

Behavioral Modeling of Power Amplifiers Based on Memory Polynomials and Lookup Tables for OFDMA and QAM Signals

André Corso Pozzan

Group of Integrated Circuits and Systems (GICS)
Federal University of Paraná (UFPR)

Curitiba, Brazil
andrepozzan@ufpr.br

Eduardo Gonçalves de Lima

Group of Integrated Circuits and Systems (GICS)
Federal University of Paraná (UFPR)

Curitiba, Brazil
eduardo.lima@ufpr.br

Abstract—The objective of this work is to reproduce and analyze models based on polynomials with memory (PM) and their variations, focusing on different identification and implementation techniques. Such models are crucial in engineering, notably applied in the behavioral modeling of power amplifiers (PAs) and in the operational predictability of components. Given the absence of a universally superior model, optimal performance depends on the specific characteristics of the PA and the input signals. The work involved variations in topology, such as the use of Volterra series, the implementation of error optimization with complex numbers, estimation of initial coefficients, digital pre-distortion to reduce nonlinear effects, and application in transmission with OFDMA and QAM modulation. The study also sought to reduce computational complexity using lookup tables (LUTs). For this, mathematical tools and simulations in Python were used. It was possible to observe different levels of precision and computational demands among the models, with the aim of applying the one that offers the best cost-benefit ratio between precision and complexity. Accuracy was evaluated using metrics such as Normalized Mean Squared Error (NMSE) by comparing the amplitude and phase difference between the input and output of the real PA and the estimated model, as well as the bit error rate (BER), used to assess the performance of the modulated transmission. By representing a component with the necessary precision, it is possible to stipulate its output behavior without the need for direct laboratory tests, optimizing time and resources.

Index Terms—memory, polynomial, volterra, ofdma, qam.

I. INTRODUCTION

Modern telecommunications demand high data rates, placing the Power Amplifier (PA) as a critical yet power-consuming component prone to signal distortion. The inherent trade-off between PA linearity and efficiency is exacerbated by memory effects and physical factors like transistor self-heating. To mitigate these distortions, behavioral modeling such as Polynomial Memory Models is widely used to mathematically represent input-output relationships [1]–[3]. These techniques are essential for OFDMA and QAM-based systems, which underpin current wireless standards [4], [5]. Extensive literature highlights OFDMA's advantages in throughput [6], [7], synchronization robustness [8], and diverse access scenarios [9]–[14].

As a main contribution, this work evaluates memory polynomials combined with Lookup Tables for Digital Pre-Distortion,

demonstrating how they significantly reduce computational overhead for OFDMA and QAM signals while maintaining accuracy. The simulated system is shown in Figure 1, focusing on PA modeling and modulation, whereas antenna and RF filter properties remain for future work.

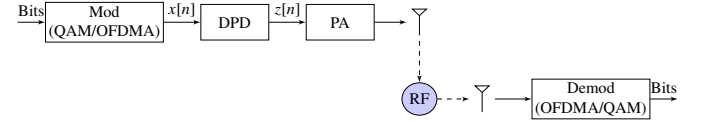


Figure 1. Complete system block diagram from Bits to recovered Bits, highlighting the DPD and PA stages.

II. MATHEMATICAL MODELING

Initially, calculations were performed with linear optimization using the least squares method (LSM) to find the coefficients of the polynomial model. The objective is to minimize the sum of squared errors between the actual output of the PA and the output estimated by the model.

It is desired to create a mathematical model for the amplifier. The chosen model is the polynomial with memory, a particular case of the Volterra series as described in [3], whose equation is:

$$\tilde{y}(n) = \sum_{p=1}^P \sum_{m=0}^M \tilde{b}_{2p-1,m} |\tilde{x}(n-m)|^{2p-2} \tilde{x}(n-m) \quad (1)$$

where $\tilde{y}(n)$ is the estimated output of the model at instant n , $\tilde{x}(n-m)$ is the input at instant $(n-m)$, $\tilde{b}_{2p-1,m}$ are the model coefficients, P is the maximum degree of the polynomial, and M is the memory depth.

Because conventional optimization algorithms do not support complex numbers, an abstraction method using a residual function is employed to convert and guide the optimization process.

To evaluate the accuracy of the model, the NMSE is given by:

$$NMSE = 10 \log_{10} \left[\frac{\sum_{n=1}^{N_1} |e(n)|^2}{\sum_{n=1}^{N_1} |y_{\text{ref}}(n)|^2} \right] \quad (2)$$

where $e(n) = y_{\text{ref}}(n) - \tilde{y}(n)$ is the error between the reference output and the estimated output, and N is the total number of samples as per [1]. The NMSE is frequently expressed in decibels (dB). A lower NMSE value (more negative in dB) indicates better model accuracy, that is, a smaller difference between the measured and estimated signals.

To mitigate the computational complexity of polynomial models, Lookup Tables (LUTs) are employed to store pre-calculated values, replacing repetitive calculations with fast data access. For complex-valued inputs and outputs, the LUT implementation utilizes linear interpolation between stored matrix elements [2], expressed as:

$$f_{\text{LUT}_m}(|\tilde{x}(n-m)|) = \tilde{s}_m(q-1) + \left[\frac{\tilde{s}_m(q) - \tilde{s}_m(q-1)}{e_m(q) - e_m(q-1)} \right] \cdot (|\tilde{x}(n-m)| - e_m(q-1)) \quad (3)$$

where the variables are defined as:

- $\tilde{s}_m(q-1)$: complex output corresponding to the input $e_m(q-1)$ (lower point);
- $\tilde{s}_m(q)$: complex output corresponding to the input $e_m(q)$ (upper point);
- $e_m(q-1)$: real input of the lower limit of the interpolation interval;
- $e_m(q)$: real input of the upper limit of the interpolation interval;
- $|\tilde{x}(n-m)|$: amplitude of the input for which the output is to be estimated.

The *lower point* is considered as that immediately below the desired value of $|x(\tilde{n}-m)|$ in the table, represented by $e_m(q-1)$, and the *upper point* as that immediately above, represented by $e_m(q)$. In this way, linear interpolation is performed with the necessary continuity conditions met.

The transmitted signal is generated using Quadrature Amplitude Modulation (QAM) mapped onto an Orthogonal Frequency Division Multiple Access (OFDMA) scheme. Each data symbol is represented in the complex plane as $X[k] = I_k + jQ_k$, where I_k and Q_k are symmetrically spaced amplitudes (e.g., $\{\pm 1, \pm 3\}$ for 16-QAM). The complex baseband signal $x[n]$ is then constructed by applying an Inverse Fast Fourier Transform (IFFT) to these subcarriers:

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[k] \exp\left(j \frac{2\pi kn}{N}\right) \quad (4)$$

where N is the IFFT size, $1/\sqrt{N}$ is the unitary normalization factor for power conservation, and the summation represents the superposition of N orthogonal modulated subcarriers.

The system performance is validated by the Bit Error Rate (BER), defined as the ratio of incorrectly received bits to the total number of transmitted bits N_2 :

$$\text{BER} = \frac{1}{N_2} \sum_{i=1}^{N_2} (b_{i,\text{tx}} \neq b_{i,\text{rx}}) \quad (5)$$

where $b_{i,\text{tx}}$ and $b_{i,\text{rx}}$ represent the i -th transmitted and received bits, respectively, and $(b_{i,\text{tx}} \neq b_{i,\text{rx}})$ is the indicator function that identifies a bit mismatch.

Basically, it's the number of bits transmitted incorrectly divided by the total number of bits transmitted.

An Automatic Gain Control (AGC) was implemented to normalize the received OFDMA+QAM signal. The scaling factor is based on the ratio between the ideal average power, P_{ideal} , and the measured power, P_{measured} , derived from the simulation. For an L -level PAM component, the energy is $E_{\text{PAM}} = (L^2 - 1)/3$. For a QAM constellation ($L = \sqrt{M}$), the ideal power and the corresponding correction gain G are defined as:

$$P_{\text{ideal}} = \frac{2}{3}(L^2 - 1) \implies G = \sqrt{\frac{P_{\text{ideal}}}{P_{\text{measured}}}} \quad (6)$$

Where P_{measured} is obtained by the mean squared magnitude of the received complex symbols.

III. PYTHON CODES

To validate the mathematical models described in the previous section, a complete simulation framework was developed. At all stages of this work, Python was employed to perform numerical calculations, system simulations, and graph generation.

The main optimization method does not work with complex numbers, to address this limitation and ensure the stability and convergence speed of the solver, the functions `unpackComplexCoefficients()` and `packComplexCoefficients()` which split complex numbers into real and imaginary parts were utilized.

Now, moving on to the main function of calculating the adjusted coefficients for amplifier modeling using LUTs, there is also the Volterra series, which was presented for comparison as described in the mathematical modeling section, however, LUTs offer better computational performance, and the focus of the code will be on them.

```
def estimatedValueWithLUT(x_data, lut_out_matrix, cfg):
    length_of_x_data = len(x_data)
    result = np.zeros(length_of_x_data, dtype=complex)
    for line in range(cfg.n_lut_lines):
        delayed = np.roll(x_data, line)
        delayed[:line] = 0
        x_abs = np.abs(delayed)
        real_interpolation = np.interp(x_abs,
        ↪ cfg.interpolation_in, lut_out_matrix[line].real)
        imag_interpolation = np.interp(x_abs,
        ↪ cfg.interpolation_in, lut_out_matrix[line].imag)
        result += delayed * (real_interpolation + 1j *
        ↪ imag_interpolation)
    return result
```

Figure 2. Function to estimate value with LUT

The Figure 3 shows the `calcCoef()` function, which uses least squares to estimate values via LUTs using the `estimatedValueWithLUT()` at the Figure 2. There is also the `residuals()`, which is necessary to help the main method converge. This allows for good modeling of the PA with computational cost-effectiveness, benefiting from the LUTs.

The optimization is performed with a nonlinear least squares solver (implemented here via SciPy), as documented in [15] and [16]

```
def residuals(lut_out_real, x_data, y_data, cfg):
    lut_out_complex = unpackComplexCoefficients(lut_out_real,
    ↪ cfg.n_lut_lines, cfg.n_lut_columns)
    ↪ y_estimated = estimatedValueWithLUT(x_data, lut_out_complex,
    ↪ cfg)
    residual = y_data - y_estimated
    res_vector = np.concatenate([residual.real, residual.imag])
    return res_vector
def calcCoef(in_data, out_data, cfg):
    initial_real_coef =
    ↪ packComplexCoefficients(cfg.initial_complex_coef)
    result = least_squares(residuals, initial_real_coef,
    ↪ args=(in_data, out_data, cfg), verbose=2)
    return unpackComplexCoefficients(result.x, cfg.n_lut_lines,
    ↪ cfg.n_lut_columns)
```

Figure 3. Functions to estimate the coefficient for PA modeling

For the digital pre-distortion method shown in the Figure 4, we use the same functions that normally calculate the PA coefficients, but now we pass the input values, to the output and vice versa to obtain the "reverse" effect, multiplying the input signal and canceling the final distortion.

```
def dpdTraining(in_training, out_training, cfg):
    dpd_coef = calcCoef(out_training, in_training, cfg)
    return dpd_coef
```

Figure 4. DPD training function

The Figure 5 represents the code developed for QAM modulation, which receives the bits, separates them into symbols, and converts them into a complex representation according to the constellation diagram.

```
def qam_mod(bits, modulation_order):
    k = int(np.log2(modulation_order))
    k_half = k // 2
    n_levels = 2**k_half
    bits = list(bits)
    if len(bits) % k != 0:
        bits += ['0'] * (k - (len(bits) % k))
    def binary_to_gray(n):
        return n ^ (n >> 1)
    levels = np.arange(n_levels)
    gray_levels = binary_to_gray(levels)
    qam_levels = 2 * np.arange(n_levels) - (n_levels - 1)
    mapping_i = np.zeros(n_levels)
    mapping_q = np.zeros(n_levels)
    for i, g in enumerate(gray_levels):
        mapping_i[g] = qam_levels[i]
        mapping_q[g] = qam_levels[n_levels - 1 - i]
    symbols = []
    for i in range(0, len(bits), k):
        bit_group = bits[i:i+k]
        bits_i = "".join(bit_group[:k_half])
        bits_q = "".join(bit_group[k_half:])
        idx_i = int(bits_i, 2)
        idx_q = int(bits_q, 2)
        comp_i = mapping_i[idx_i]
        comp_q = mapping_q[idx_q]
        symbols.append(complex(comp_i, comp_q))
    return np.array(symbols)
```

Figure 5. Function to QAM modulation

In this context, the Figure 6 shows the function that generates

the OFDMA signal using the QAM modulation code. For the purposes of this work, random bits were generated to simulate transmission; however, a real data transmission could be implemented. Finally, the inverse Fourier transform is performed to transform the signal from the frequency domain to the time domain for transmission.

```
def generate_OFDMA_signal(Nfft, number_of_users,
    ↪ subcarriers_per_user, modulation_order):
    complete_band = np.zeros(Nfft, dtype=complex)
    all_original_bits = []
    for user in range(number_of_users):
        start = user * subcarriers_per_user
        end = start + subcarriers_per_user
        bits = ''.join(np.random.choice(['0', '1'],
        ↪ int(np.log2(modulation_order)) * subcarriers_per_user))
        bits = bits.replace(" ", "")
        symbols = qam_mod(bits, modulation_order)
        complete_band[start:end] = symbols
        all_original_bits.append(bits)
    band_ifft = np.fft.ifft(complete_band) * np.sqrt(Nfft)
    return band_ifft, all_original_bits
```

Figure 6. Function to generate OFDMA signal

GitHub repository with all the codes made during scientific initiation:

github.com/andrepozzan/ic-gics

IV. RESULTS

The signals analyzed in this work were obtained from a Class AB power amplifier produced with GaN technology. We employed a carrier centered at 900 MHz, modulated by an envelope signal suitable for WCDMA (with a bandwidth close to 3.84 MHz). Input and output waveforms were recorded using a Rohde & Schwarz FSQ vector signal analyzer, operating at a sampling rate of 61.44 MHz. For the experiments, the data were divided into two sets: 3,221 samples for extraction (training) and 2,001 samples for model validation.

This simulation considers two users with 8 subcarriers each, with 16-QAM modulation and an NFFT size of 2048. A 4×4 LUT matrix was used for the coefficient adjustment. This code is generalized to work with larger values, but for the purposes of this presentation, a simpler case was chosen.

For digital pre-distortion training, the graph in the Figure 7 clearly shows the "reverse" effect that DPD applies, resulting in amplifier linearization.

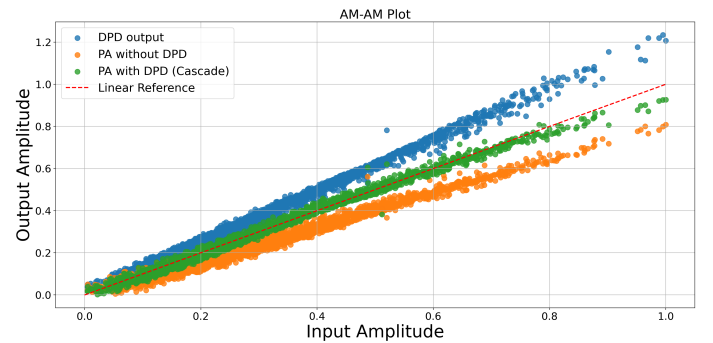
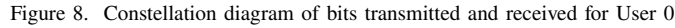


Figure 7. Digital pre-distortion result in AM-AM

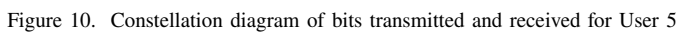
Now, to verify the signal demodulation, the original and demodulated bits are plotted on each user's constellation

For a simplified analysis and easy visualization of the process to converting bits into symbols and plotting them on constellation diagram can be observed in Figure 8.



```
qam_levels: [-3 -1 1 3] gain: 0.5993523460988588
i_levels: [1 1 -1 -1 1 3 3 -1]
q_levels: [-1 -1 3 1 3 -1 -3 1]
Bits send User 0: 1111 1111 0100 1101 1100 1011 1010 0101
Bits received User 0: 1111 1111 0100 1101 1100 1011 1010 0101
User 0: BER = 0.00e+00 (0/32 )
```

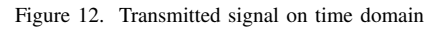
In this context, modern communication systems handle a higher volume of users and data, leading to more complex constellation diagrams and higher error rates, however, the same principle applies.



and presented in Figure 11, demonstrating the performance of the system under these conditions.

User 0: BER = 11.92% (31/260)	User 1: BER = 14.23% (37/260)
User 2: BER = 13.85% (36/260)	User 3: BER = 7.31% (19/260)
User 4: BER = 16.15% (42/260)	User 5: BER = 7.31% (19/260)
User 6: BER = 13.08% (34/260)	User 7: BER = 5.77% (15/260)

The Figure 12 shows the time-domain signal carrying the bit information from two users. It can be observed that the amplifier with pre-distortion closely follows the original theoretical signal. This signal, for example, could be applied to an RF transmission circuit, resulting in a prototype wireless router.



V. CONCLUSION

From this perspective, the results highlight the practical viability of the proposed approach for modern communication systems. Future research will focus on evaluating alternative circuit topologies to maximize throughput under complex modulation formats. Furthermore, a sensitivity analysis under varying SNR and memory depths (M) is essential to evaluate model robustness. Investigating diverse optimization algorithms also presents an opportunity to refine the trade-off between convergence speed and estimation accuracy.

REFERENCES

- [1] E. G. de Lima, “Behavioral modeling and digital base-band predistortion of RF power amplifiers”, Ph.D. dissertation, POLITECNICO DI TORINO, Jan. 2009.
- [2] C. L. R. Machado, “Modelagem comportamental de amplificadores de potência usando soma de produtos entre filtros digitais de resposta ao impulso finita e tabelas de busca unidimensionais”, M.S. thesis, Universidade Federal do Paraná, Curitiba, 2016. [Online]. Available: <https://acervodigital.ufpr.br/handle/1884/46250>.
- [3] E. J. Bonfim, “Modelagem comportamental de amplificadores de potência de radiofrequência usando termos unidimensionais e bidimensionais de séries de volterra”, M.S. thesis, Universidade Federal do Paraná, Curitiba, 2016. [Online]. Available: https://btd.ibict.br/vufind/Record/UFPR_54e3f81abd9cae4e13508c3e91186f38.
- [4] 3GPP, 5g; nr; physical channels and modulation, https://www.3gpp.org/ftp/Specs/archive/38_series/, 2022.
- [5] Huawei, *Ofdma*, <https://info.support.huawei.com/info-finder/encyclopedia/en/OFDMA.html>, Huawei Technologies Co., Ltd., 2024.
- [6] A. Nusairat and X.-Y. Li, “Wimax/ofdma burst scheduling algorithm to maximize scheduled data”, *IEEE Transactions on Mobile Computing*, vol. 11, no. 11, pp. 1692–1705, 2012. DOI: [10.1109/TMC.2011.211](https://doi.org/10.1109/TMC.2011.211).
- [7] H. Bany Salameh, H. Al-Obiedollah, R. Mahasees, and Y. Jararweh, “Opportunistic non-contiguous ofdma scheduling framework for future b5g/6g cellular networks”, *Simulation Modelling Practice and Theory*, vol. 119, p. 102563, 2022, ISSN: 1569-190X. DOI: [10.1016/j.simpat.2022.102563](https://doi.org/10.1016/j.simpat.2022.102563). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1569190X22000569>.
- [8] Z. Cao, U. Tureli, and Y.-D. Yao, “Low-complexity orthogonal spectral signal construction for generalized ofdma uplink with frequency synchronization errors”, *IEEE Transactions on Vehicular Technology*, vol. 56, no. 3, pp. 1143–1154, 2007. DOI: [10.1109/TVT.2007.895601](https://doi.org/10.1109/TVT.2007.895601).
- [9] A. Mutairi, S. Roy, and G. Hwang, “Delay analysis of ofdma-aloha”, *IEEE Transactions on Wireless Communications*, vol. 12, no. 1, pp. 89–99, 2013. DOI: [10.1109/TWC.2012.113012.111776](https://doi.org/10.1109/TWC.2012.113012.111776).
- [10] G. Berardinelli, L. A. M. Ruiz de Temino, S. Frattasi, M. I. Rahman, and P. Mogensen, “OFDMA vs. SC-FDMA: Performance comparison in local area imt-a scenarios”, *IEEE Wireless Communications*, vol. 15, no. 5, pp. 64–72, 2008. DOI: [10.1109/MWC.2008.4653134](https://doi.org/10.1109/MWC.2008.4653134).
- [11] P. W. Chan, E. S. Lo, V. K. Lau, *et al.*, “Performance comparison of downlink multiuser mimo-ofdma and mimo-mc-cdma with transmit side information - multi-cell analysis”, *IEEE Transactions on Wireless Communications*, vol. 6, no. 6, pp. 2193–2203, 2007. DOI: [10.1109/TWC.2007.05518](https://doi.org/10.1109/TWC.2007.05518).
- [12] X. Guo and Y. Luo, “Hybrid noma/ofdma visible light communication system with coordinated multiple point transmission”, *Opt. Express*, vol. 30, no. 26, pp. 47404–47420, Dec. 2022. DOI: [10.1364/OE.477769](https://doi.org/10.1364/OE.477769). [Online]. Available: <https://opg.optica.org/oe/abstract.cfm?URI=oe-30-26-47404>.
- [13] S. Althunibat, R. Mesleh, and K. A. Qaraqe, “Im-ofdma: A novel spectral efficient uplink multiple access based on index modulation”, *IEEE Transactions on Vehicular Technology*, vol. 68, no. 10, pp. 10315–10319, 2019. DOI: [10.1109/TVT.2019.2937080](https://doi.org/10.1109/TVT.2019.2937080).
- [14] S. Avallone, P. Imputato, G. Redieteb, C. Ghosh, and S. Roy, “Will ofdma improve the performance of 802.11 wifi networks?”, *IEEE Wireless Communications*, vol. 28, no. 3, pp. 100–107, 2021. DOI: [10.1109/MWC.001.2000332](https://doi.org/10.1109/MWC.001.2000332).
- [15] SciPy, *Scipy.optimize.least_squares: Solve a nonlinear least-squares problem with bounds on the variables*, Accessed on: 20 abr. 2025, 2025. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html.
- [16] MathWorks, *Lsqnonlin: Solve nonlinear least-squares (nonlinear data-fitting) problems*, Accessed on: Apr. 1, 2025, 2025. [Online]. Available: <https://www.mathworks.com/help/optim/ug/lsqnonlin.html>.